

Pittsburgh, Pennsylvania 412.287.8640 russelldudek@gmail.com linkedin.com/in/russelldudek github.com/russelldudek/Quick

ENTRY PLAN

Read the system. Establish continuity. Earn scale.

Day-90 objective

Produce one evidence-backed production improvement and a reusable Enterprise AI Backplane practice without pretending the undisclosed client environment is already known.

Operating assumptions to verify immediately

Role center of gravity

Confirm the balance among model development, ML platform engineering, enterprise integration, technical leadership, and customer-facing solution work.

Current production estate

Identify deployed systems, clouds, frameworks, pipelines, APIs, data sources, evaluation approaches, monitoring, and incident history.

Decision rights

Clarify architecture authority, release approval, risk escalation, production ownership, support ownership, and retirement authority.

Success measures

Agree which workflow outcomes, reliability signals, costs, controls, adoption evidence, and learning measures matter.

Three-phase operating sequence

Phase	Primary question	Direct work	Exit evidence
0-30: Trace	What system are we actually operating?	Map architecture, work, data, APIs, evaluation, controls, incidents, economics, and owners.	System map, baseline, failure taxonomy, Backplane Review target.
31-60: Harden	Which interface limits production confidence?	Close one control gap; add tests, observability, fallback, rollback, and decision ownership.	Bounded proof plan and hardening evidence.
61-90: Compound	What survived and should become reusable?	Observe, learn, document, standardize, and recommend portfolio disposition.	Scorecard, reusable pattern, ownership model, scale/hold decision.

Principles

Work backward from consequence

Define the workflow, user, baseline, risk, and decision before architecture.

Fix the weakest interface

Do not optimize the model while context, runtime, authority, or evidence remains the system limit.

Harvest every proof

Each release should leave reusable architecture, tests, evidence, or a fast no.

412.287.8640 russelldudek@gmail.com linkedin.com/in/russelldudek github.com/russelldudek/Quick

DAYS 0-30

Trace the production reality.

Portfolio and use-case map

Document active, piloting, queued, paused, and retired systems; users; workflows; outcomes; sponsors; risk class; and technical owners.

Architecture and dependency map

Trace models, prompts, retrieval, data stores, APIs, services, queues, cloud resources, authentication, security controls, and downstream actions.

Evaluation and telemetry map

Inspect offline evaluations, production metrics, task evidence, human feedback, drift, incidents, cost visibility, and alert ownership.

Authority and operating map

Identify approvals, overrides, exception handling, escalation, release gates, rollback authority, support, and change cadence.

Baseline evidence

Domain	Questions	Output
Outcome	Which decision or work changes? What is the current behavior and consequence?	Named outcome, owner, baseline, target, stop condition.
Context	Which sources are authoritative? How are freshness, quality, versions, access, and conflict handled?	Source/provenance map and readiness exceptions.
Model	How was the model selected? Which cases, thresholds, error types, and fallbacks are tested?	Evaluation inventory and gap list.
Runtime	What are dependency, latency, throughput, availability, cost, degradation, and recovery patterns?	Reliability envelope and incident baseline.
Authority	Which actions are advisory, reviewed, approved, overridden, escalated, or prohibited?	Decision-rights and exception map.
Evidence	Who reviews technical health, usefulness, adoption, incidents, drift, and retirement?	Scorecard and ownership gaps.

DAYS 31-60

Close the weakest interface.

Select a bounded proof

Choose one meaningful workflow whose friction, consequence, ownership, and technical boundary are visible enough to measure.

Write the six contracts

Make outcome, context, model, runtime, authority, and evidence explicit.

Harden one fault path

Add the right test, monitoring, fallback, timeout, escalation, approval, rollback, or documentation.

Define acceptance

Specify functional, quality, reliability, cost, control, adoption, and recovery evidence.

Run a production drill

Exercise context drift, dependency failure, authority conflict, or quality decay.

Prepare the decision

State what evidence will support scale, further bounding, hold, rollback, or stop.

412.287.8640 russelldudek@gmail.com linkedin.com/in/russelldudek github.com/russelldudek/Quick

DAYS 61-90

Turn the proof into engineering leverage.

Observe the bounded release

Track output quality, task outcome, latency, availability, cost, user behavior, overrides, incidents, and recovery.

Run the learning loop

Review false confidence, failure modes, support burden, user feedback, control exceptions, and ownership friction.

Codify reusable interfaces

Create architecture notes, evaluation templates, fault tests, control patterns, telemetry expectations, documentation, and review criteria.

Recommend portfolio disposition

Scale, further bound, hold, roll back, or retire. State evidence, unresolved risk, economics, ownership, and the next decision date.

Day-90 operating scorecard

Domain	Illustrative measures	Decision question
Outcome	Task quality, completion, cycle time, first-pass quality, rework.	Does the system improve the intended work?
Reliability	Latency, throughput, availability, dependency failures, recovery, rollback.	Can the workflow depend on it within a known envelope?
Economics	Cost per useful outcome, human effort, infrastructure, support burden.	Is value credible after operating cost?
Control	Authority coverage, exceptions, policy adherence, auditability.	Are high-consequence actions bounded?
Adoption	Retained use, task penetration, overrides, workarounds, trust.	Has capability become normal work?
Learning	Evaluation coverage, drift, incident recurrence, feedback-to-change time.	Does the system improve faster than risk accumulates?
Ownership	Named model, source, runtime, workflow, outcome, incident, and retirement owners.	Can the right person make the next decision?

Risks to surface early

Role mismatch

The position may be primarily novel model research.

Access without authority

A staff contributor may see risk but lack decision rights to close it.

Metric substitution

Offline scores, uptime, or usage can obscure poor workflow outcomes.

One-off success

A proof may work without producing reusable architecture or practice.

Ownerless handoff

Technical delivery can precede operating ownership.

Scale before recovery

Reach should not expand faster than monitoring, rollback, and incident learning.